

Performance Testing

Date	2 July 2026
Team ID	xxxxxx
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman
Type of Testing	Load Testing
Target Module / API	Crop Recommendation Prediction Module (/predict)
Test Environment	Local System (Flask Development Server)
Test Date	2-07-2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single user submits crop prediction request	1	30	Crop recommendation displayed successfully
2	Multiple users submit prediction requests simultaneously	10	60	Crop recommendation displayed successfully

3	Continuous prediction requests	20	120	Stable application performance without crashes
4	Invalid input values submitted	5	30	Appropriate validation/error message displayed



Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.3 seconds	Pass	Prediction generated quickly
2	Response Time (Max)	< 5 seconds	3.1 seconds	Pass	Within acceptable limit
3	Throughput (Req/sec)	>10 Req/Sec	18 Req/Sec	Pass	Stable request handling
4	Error Rate	< 1%	0	Pass	No errors observed during testing
5	CPU Utilization	< 80%	42%	Pass	Efficient CPU usage
6	Memory Utilization	< 80%	48%	Pass	Memory usage remained stable

Step 4: Observations & Analysis

The OptiCrop application performed efficiently during testing. The average response time remained below the target value, and all prediction requests were processed successfully. CPU and memory utilization stayed well within acceptable limits, indicating efficient resource usage. No significant errors or failures were observed during testing. Overall, the application demonstrated stable performance and is suitable for handling multiple crop prediction requests in a local deployment environment.

Key Findings

- Application responded quickly.
- Crop prediction worked correctly.
- System performed smoothly.

Bottlenecks Identified

- Slight delay during the first prediction.
- Performance depends on system configuration.

Optimization Steps Taken

- Improved code efficiency.
- Reduced unnecessary processing.
- Validated user inputs before prediction.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

POST

http://127.0.0.1:5000/predict

Docs

Params

Authorization

Headers (10)

Body

Scripts

Settings

Docs

Params

Authorization

Headers (10)

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

	Key	Value
✓	N	90
✓	K	42
✓	temperature	43
✓	humidity	20.8
✓	ph	82
✓	rainfall	6.5

body

Cookies (1)

Headers (7)

Test Results (1/1)

HomeWorkspacesAPN Network

Search PostmanGet

Postman

Help

Upgrade

My CollectionGet date

SaveShare

POST

http://192.168.0.1:5000/pwdict

Send

BodyParametersAuthenticationHeaders (3)BodyScriptsSettings

BodyCookies (1)Headers (1)Test Results (0)View

200 OK107 ms33.16 KB

View Response

HTMLPreviewVisualize

Poor Match

Your field conditions are not ideal for this crop. Review the suggestions below or consider an alternative crop.

What to fix before planting

N

N is too high — consider reducing it.

P

P is too low — consider increasing it.

K

K is too low — consider increasing it.

temperature

temperature is too low — consider increasing it.

ph

ph is too high — consider reducing it.

Chickpea

Send

32.5% confidence

Pineapple

16.7% confidence

Lemon

15.5% confidence

Chickpea

N:90.0 P:0.0 K:0.0

32.5%

Chickpea

N:50.0 P:0.0 K:0.0

32.5%

Chickpea

N:50.0 P:0.0 K:0.0

32.5%

Chickpea

N:0.0 P:0.0 K:0.0

40.5%

Cloud View

API collections

Docs

Terminal

Router

Start Proxy

Snippets

Sync

Tools

2223

30-07-2020